COMP 356 Sample Midterm Exam II

1. (7 points) Why is is straightforward to build dynamic data structures using references in Java, but difficult to do so using references in C++?

2. Consider the following C++ program:

```
int main() {
  int *p1, *p2, *p3;
  int i = 3;

  p1 = new int;
  p3 = new int;
  p2 = &i;
  p1 = p3;
  delete(p1);
}
```

   (a) (7 points) Does this program create any garbage? Why or why not?

   (b) (5 points) List all pointers that are dangling after the execution of `delete(p1);`.

3. (6 points) Suppose that for a particular C++ compiler, the size of an `int` in memory is 2 bytes, of a `float` is 4 bytes and of a `double` is 8 bytes. How much memory is required for a variable of the following type `utype`?

```
typedef union {
  double d;
  struct {
    int i;
    double d;
  } s1;
  struct {
    int j;
    int k;
    float f;
  } s2;
} utype;
```

4. Consider the following program:

```
#include <iostream.h>

void foo(int a, int b) {
  a = 3;
  a = a + b;
  b = a + 2;
}

int main() {
  int x = 5;

  foo(x, x);
  std::cout << x << std::endl;
}
```

What output does this program produce under each of the following parameter passing mechanisms? Show your work.

- (4 points) call-by-value

- (8 points) call-by-reference

- (8 points) call-by-value-result (assume that the copy-outs occur in parameter order, i.e. first `a`, then `b`)

5. Consider the following C++ macro definition:

```
#define foo(x, y) (x = x * y)
```

For each of the following calls of this macro, either state that the call would cause an error at compile time, or if the call would not cause an error, give the value of variable `i` resulting from the call. The initial value of `i` is `3` for each call.

(a) (5 points) `foo(i, i + 2);`

(b) (5 points) `foo(i + 2, i);`

6. (10 points) Complete the definition of the following C++ function `add`, which adds two 5 by 5 matrices of integers. The sum of the matrices should be placed into the first parameter (so that the original value of this parameter is lost). The matrices are represented by two dimensional arrays of integers. You are NOT allowed use square brackets (`[ ]`) to index into either of the array parameters in any of the code that you write.

```
void add(int m1[][5], int m2[][5]) {
  int *p1 = &(m1[0][0]);
  int *p2 = &(m2[0][0]);









}
```

7. (8 points) Write a tail recursive version of the following C function `power`, which computes x^n. Include an "interface" function that calls your tail recursive function with the appropriate initial value for the accumulator parameter.

```
int power(int x, int n) {
  if (n < 1) return 1;
  else return x * power(x, n - 1);
}
```

8. Consider the following program:

```
#include <iostream>
int x = 3, y = 5;

void bar(void) {

  y = y + 2;
  std::cout << x << " " << y << std::endl;
}

void baz(void) {
  int y = 7;
  x = x + 2;
  bar();
  std::cout << y << std::endl;
}

int main() {
  int x = 5;
  x = x + 2;
  baz();
  std::cout << x << std::endl;
}
```

What output does this program produce under each of the following scoping rules?

(a) (4 points) static scope

(b) (6 points) dynamic scope